

Семинар №8

Move-семантика.
Продолжение

Передача объекта дальше

- Мы умеем делаем перегрузку, разделяя `const T&/T&&`
- Также часто тело перегружаемых функции одинаково с точность до `std::move`
- Есть ли способ объединить две перегрузки в одну?
- Особенно остро стоит проблема с `variadic templates` в качестве аргументов, потому что каждый аргумент может передаваться по `const T&` или по `T&&` (например, для методов `emplace`)

std::forward

- Принимаем аргументы с помощью Args&&, где Args – шаблонный параметр функции / метода
- Передаем аргументы с помощью std::forward<Args>(args)...
- Сохраняется “нужный” вид ссылки (lvalue, rvalue), то есть тот, который был при вызове внешней функции
- Но как мы можем принимать lvalue, если имеем Args&&?

Универсальные ссылки

- Ссылка `Args&&` называется универсальной, если
 - 1) `Args` является шаблонным параметром функции/метода (но не целого класса)
 - 2) Принимается аргумент **ровно** `Args&&` (не `const Args&&`, `std::vector<Args&&>`, ...)
 - 3) Аргумент функции

Если тип шаблонного параметра указан явно, то правила вывода типов не применяются

Type deduction

(без универсальных ссылок, напоминание)

```
template <typename T>  
void f(ParamType param);  
f(expr);
```

Типы ParamType и тип expr
могут не совпадать

Если ParamType является указателем или ссылкой:

1. Если тип expr это ссылка, игнорируем ссылочность
2. Паттерн-матчинг типа expr в ParamType для выяснения T

Если ParamType **не** является ни указателем, ни ссылкой:

1. Если тип expr это ссылка, игнорируем ссылочность (как и раньше)
2. Игнорируем константность тоже (так как всё равно создаётся копия объекта)

Type deduction (с универсальными ссылками)

TL;DR:

Ссылочность всегда игнорируется, кроме случая универсальных ссылок и lvalue

```
template <typename T>  
void f(ParamType param);  
f(expr);
```

Если ParamType является указателем или ссылкой (**но не универсальной ссылкой**):

1. Если тип expr это ссылка, игнорируем ссылочность
2. Паттерн-матчинг типа expr в ParamType для выяснения T

Если ParamType является **универсальной ссылкой**:

1. Если категория expr – lvalue, то T выводится как **lvalue-reference**
2. Если категория expr – rvalue, то применяются правила выше

Если ParamType **не** является ни указателем, ни ссылкой:

1. Если тип expr это ссылка, игнорируем ссылочность (как и раньше)
2. Игнорируем константность тоже (так как всё равно создаётся копия объекта)

Reference collapsing

- Умеем выводить тип шаблонного параметра, а как вывести тип самого аргумента?
- $_ + \& \rightarrow \&$
- $_ + \&\& \rightarrow \&\&$
- $\& + \& \rightarrow \&$
- $\&\& + \& \rightarrow \&$
- $\& + \&\& \rightarrow \&$
- $\&\& + \&\& \rightarrow \&\&$

std::move & std::forward

- Если бы std::move возвращал T&&, то мог бы происходить reference collapsing -> заменяем на std::remove_reference_t<T>&&
- std::move должен уметь принимать как rvalue, так и lvalue, поэтому он принимает универсальную ссылку
- std::forward принимает std::remove_reference_t<T>&, чтобы запретить неявный вывод шаблонных параметров
- Перегрузка std::forward принимает std::remove_reference_t<T>&&, чтобы уметь принимать не только std::forward<Args>(args), но и std::forward<Args>(foo()), где foo возвращает rvalue

Copy elision

- Не создаём лишние копии объекта, когда это не обязательно, даже если могли бы быть side эффекты
- `return obj`, где `obj` – временный объект (не аргумент), того же типа, что и возвращаемое значение (NRVO)
- `throw obj`
- `prvalue` не материализуется, пока это не является необходимым, и создаётся сразу на месте “нужного” объекта
- Если `initializer expression` является `prvalue` того же типа:
 - `T x = T(T(f()));` // `T` инициализируется сразу результатом `f`
- [Подробнее](#)

RVO (return value optimization)

- Если возвращается не локальная переменная, то RVO не срабатывает, но всё равно переменная перемещается.
- Не нужно писать `return std::move(локальный объект)`
- Однако, если перемещается аргумент, принятый по rvalue ссылке, то для него есть смысл писать `return std::move`, потому что иначе копирование